

Normalizador Estadístico Vectorizado: Comparación de Rendimiento entre Implementaciones Escalar y Vectorial (AVX2) en Ensamblador x86-64

Wilbert Alonso Román Rodríguez
Carnet: 2019042296

Victoria Slyn Cerdas López
Carnet: 2023113288

Abstract—Este informe documenta el diseño, implementación y evaluación de rendimiento de un normalizador estadístico (media, varianza, desviación estándar, mínimo, máximo y normalización z-score) implementado en dos versiones de ensamblador x86-64: una escalar y una vectorizada con AVX2 (instrucciones de 256 bits, 8 floats por operación). Ambas versiones se enlazan a un programa controlador (*driver*) en C mediante la convención de llamada System V AMD64 ABI. Se presenta la metodología de verificación de correctud, el manejo de casos borde, la medición de tiempos de ejecución para distintos tamaños de entrada, y un análisis de rendimiento basado en `perf stat` que explica por qué el *speedup* observado no alcanza el límite teórico de 8x.

Index Terms—NASM, ensamblador x86-64, AVX2, SIMD, System V ABI, rendimiento, normalización z-score

I. INTRODUCCIÓN Y OBJETIVOS

El presente proyecto tiene como objetivo comparar el desempeño de dos implementaciones de un mismo algoritmo estadístico, escrita en ensamblador escalar y otra vectorizada con la extensión AVX2, para cuantificar de forma empírica el beneficio de la programación SIMD (*Single Instruction, Multiple Data*) [1].

Los objetivos específicos son:

- Implementar, en NASM, las funciones `sum_array`, `compute_stats` y `normalize_array` en su versión escalar y vectorial, respetando la convención de llamada System V AMD64 ABI [2] para su interoperabilidad con un *driver* en C.
- Verificar la correctud de ambas implementaciones contra una referencia en Python/NumPy, incluyendo el manejo explícito de casos borde ($N=0$, $N=1$, N no múltiplo de 8, varianza cero, valores negativos y extremos).
- Medir y comparar el tiempo de ejecución de ambas versiones para distintos tamaños de entrada, calculando el *speedup* y analizando su comportamiento mediante contadores de hardware (`perf stat`).

La documentación gráfica detallada de la arquitectura, flujos de control, tablas de registros y organización de memoria se entrega mediante archivos independientes en formato PDF ubicados en el directorio `Docs/` del repositorio:

- Arquitectura de software: `Docs/Diagrama de bloques de la arquitectura de software.pdf`, detallando el enlace de módulos y convenciones de la System V AMD64 ABI.

- Flujo de control escalar: `Docs/Diagrama de flujo de control_scalar.pdf`, ilustrando el bucle de procesamiento secuencial SISD.
- Flujo de control vectorial: `Docs/Diagrama de flujo de control_AVX2.pdf`, que especifica el bucle de 8 elementos, reducción horizontal y bucle remanente (*tail loop*).
- Alineación y mapa de memoria: `Docs/Diagrama de memoria.pdf`, mostrando la base alineada a 32 bytes y los desplazamientos por iteración.
- Asignación de registros: `Docs/Tablas de Asignacion de Registros(scalar).pdf` y `Docs/Tablas de Asignacion de Registros AVX2.pdf`.

II. ENTORNO DE PRUEBAS

Todas las pruebas se ejecutaron en el siguiente entorno:

- **CPU:** 13th Gen Intel(R) Core(TM) i7-13620H (arquitectura híbrida con núcleos *performance* y *efficient*).
- **Soporte AVX2:** confirmado mediante `lscpu | grep avx2` (ver Figura 1); el flag `avx2` aparece explícitamente en la lista de Flags reportada por `lscpu`.
- **Sistema operativo:** Ubuntu 24.04.4 LTS (*noble*).
- **NASM:** versión 2.16.01.
- **GCC:** versión 13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1).
- **GDB:** GNU gdb (Ubuntu 15.1-1ubuntu1 24.04.1) 15.1, visible en el encabezado de la sesión de depuración de la Sección VII (Figura 9).

Como la CPU de prueba es híbrida (núcleos *performance*, o `cpu_core`, y *efficient*, o `cpu_atom`), los contadores de `perf stat` se reportan por separado para cada tipo de núcleo (véase la Sección VI-A); esto añade variabilidad a las mediciones de tiempo si el planificador del sistema operativo migra el proceso entre ambos tipos de núcleo durante una corrida.

III. IMPLEMENTACIÓN ESCALAR

A. Función `sum_array`

La implementación utiliza un bucle escalar secuencial. Inicia limpiando el registro acumulador `xmm0` con la instrucción `xorps` para evitar el arrastre de basura en memoria. En

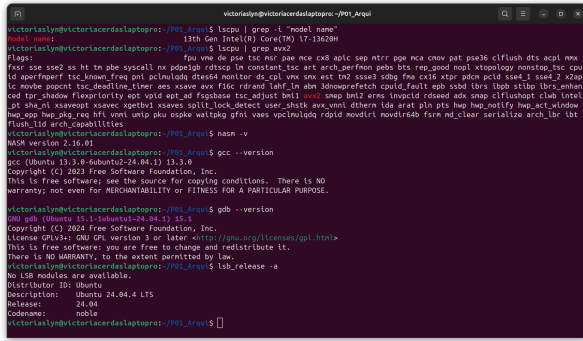


Fig. 1: Confirmación de entorno de pruebas y soporte AVX2

cada iteración, carga un único valor flotante de 32 bits desde memoria hacia el registro `xmm1` usando direccionamiento indexado (`[rdi + rax*4]`) y lo suma al acumulador mediante `addss`. El ciclo es controlado por el contador `eax`, comparándolo contra el tamaño de la muestra N en `esi`.

Listing 1: Bucle principal de `sum_array`

```
1 .sum_loop:
2     cmp     eax, esi           ; Condicion de paro del
3     jge     .sum_done         ; Salida del bucle
4     movss   xmm1, [rdi + rax*4] ; Direccionamiento
5     addss   xmm0, xmm1        ; Acumular en xmm0
6     inc     eax               ; i++
7     jmp     .sum_loop         ; Siguiente iteracion
```

El flujo de ejecución secuencial y la condición de parada del bucle escalar se detallan en el diagrama complementario Docs/Diagrama de flujo de control_scalar.pdf. Asimismo, el ciclo de vida y la ocupación de los registros involucrados (`rdi`, `esi`, `rcx` y `xmm0`) se encuentran documentados en Docs/Tablas de Asignación de Registros (scalar).pdf.

B. Función `compute_stats`

El diseño respeta estrictamente la convención *System V AMD64 ABI*, resguardando los registros *callee-saved* en la pila durante el prólogo. El caso borde donde $N \leq 0$ se maneja con una evaluación condicional (`cmp esi, 0`) al inicio; si se cumple, salta a la etiqueta `.handle_zero`, escribiendo 0.0 en los punteros de salida para prevenir divisiones por cero. En el flujo normal, invoca a `sum_array` para calcular la media poblacional, y luego realiza un segundo recorrido sobre el arreglo. En este segundo ciclo, actualiza el mínimo y máximo empleando las instrucciones dedicadas `minss` y `maxss`, y acumula la varianza calculando $(x_i - \mu)^2$ paso a paso.

Listing 2: Acumulación de varianza y actualización de extremos en `compute_stats`

```
1 .stats_loop:
2     cmp     eax, r13d          ; i == n?
3     jge     .stats_done
4     movss   xmm5, [r12 + rax*4] ; arr[i]
5
6     minss   xmm3, xmm5         ; Actualizar minimo con
7     maxss   xmm4, xmm5         ; Actualizar maximo con
```

```
7     maxss   xmm4, xmm5         ; Actualizar maximo con
8     instruction SSE
9
10    subss    xmm5, xmm0         ; (arr[i] - mean)
11    mulss    xmm5, xmm5         ; (arr[i] - mean)^2
12    addss    xmm2, xmm5         ; var_acc += (arr[i] - mean)^2
13
14    inc      eax
15    jmp     .stats_loop
```

C. Función `normalize_array`

Se procesa el arreglo aplicando la normalización estadística $y_i = (x_i - \mu)/\sigma$. Para proteger el sistema frente a arreglos de datos constantes (donde la desviación estándar $\sigma = 0$), se emplea la instrucción `comiss` para comparar la desviación (`xmm1`) contra 0.0. Si se detecta este caso borde, la instrucción de salto condicional `je` redirige el flujo a `.stddev_zero`. Este bucle alterno evade la división por cero copiando de forma directa el valor original x_i en y_i utilizando `movss`.

Listing 3: Validación del caso borde $\sigma = 0$ en `normalize_array`

```
1     xorps   xmm2, xmm2         ; xmm2 = 0.0
2     comiss   xmm1, xmm2         ; Comparamos stddev (xmm1) con
3     0.0 (xmm2)
4     je      .stddev_zero       ; Si es cero, saltamos a la
5     rutina especial
```

IV. IMPLEMENTACIÓN VECTORIAL (AVX2)

Se eligió AVX2 sobre SSE por procesar el doble de elementos por instrucción (8 floats de 32 bits en un registro YMM de 256 bits, frente a 4 en un registro XMM de 128 bits con SSE), maximizando el paralelismo de datos disponible en la CPU de prueba. Se descartó AVX-512 por no estar soportado en el hardware de prueba y por su menor disponibilidad en procesadores de consumo general [1].

Para evitar penalizaciones por cruce de líneas de caché (*cache line split*) y habilitar el rendimiento óptimo del conjunto de instrucciones AVX2, los arreglos de entrada y salida se alojan con alineación estricta a 32 bytes en memoria mediante `aligned_alloc`. La disposición física de los datos y el contraste de desplazamiento de punteros (paso de 4 bytes en el modo escalar frente a 32 bytes en el modo vectorial) se representan en el archivo complementario Docs/Diagrama de memoria.pdf.

A. Función `sum_array`

Provista como ejemplo en el esqueleto del curso. Acumula 8 sumas parciales en `ymm2` a lo largo del arreglo, y al finalizar el bucle principal aplica una reducción horizontal para obtener un único escalar, seguida de un bucle escalar de cierre para los elementos que no completan un bloque de 8 (el remanente, cuando N no es múltiplo de 8).

B. Función `compute_stats`

Se implementa en dos pasadas sobre el arreglo:

Pasada 1 – media: se reutiliza `sum_array` y se divide el resultado entre N .

Pasada 2 – varianza, mínimo y máximo: para cada bloque de 8 elementos se calcula $(x - \mu)^2$ con `vsubps`+`vmulps`,

acumulándolo con `vaddps`; en paralelo se actualizan los acumuladores de mínimo y máximo con `vminps/vmaxps`. Al salir del bucle vectorial, los tres acumuladores (suma de cuadrados, mínimo, máximo) se reducen horizontalmente de 8 carriles a 1 escalar, y los elementos sobrantes ($N \bmod 8$) se procesan con instrucciones escalares equivalentes (`vsubss`, `vmulss`, `vaddss`, `vminss`, `vmaxss`).

Listing 4: Reducción de varianza dentro de `compute_stats` (AVX2)

```
1 .cs_reduce:
2   ; reduccion horizontal de la suma de cuadrados
3   vextractf128 xmm7, ymm2, 1
4   vaddps xmm2, xmm2, xmm7
5   vhaddps xmm2, xmm2, xmm2
6   vhaddps xmm2, xmm2, xmm2 ; xmm2[0] = suma total de
   cuadrados
```

Se justifica el uso de `vextractf128 + vhaddps` para la reducción (en lugar de una secuencia de `vshufps`) por su menor número de instrucciones para reducir un acumulador de suma en 8 carriles, aunque para mínimo/máximo se optó por `vshufps` dado que `vhaddps` solo define reducción por suma.

El caso borde $N = 0$ se maneja al inicio de la función, escribiendo `0.0` en los cuatro punteros de salida sin ejecutar ninguna división, evitando así una división por cero.

C. Función `normalize_array`

Aplica $y_i = (x_i - \mu)/\sigma$ de forma vectorizada, cargando 8 elementos a la vez, restando la media difundida con `vbroadcastss` y dividiendo entre la desviación estándar también difundida. El caso borde $\sigma = 0$ (arreglo con todos los valores iguales) se detecta con `vucomiss` antes de entrar al bucle principal; en ese caso se copia la entrada a la salida sin alteración, evitando una división por cero.

Listing 5: Manejo del caso $\sigma = 0$ en `normalize_array`

```
1 vxorps xmm4, xmm4, xmm4
2 vucomiss xmm1, xmm4 ; compara stddev contra 0.0
3 je .na_copy ; si stddev == 0.0 -> copiar
   tal cual
```

D. Manejo del remanente

Las tres funciones siguen el mismo patrón estructural: un bucle vectorial que avanza de 8 en 8 mientras $i < N - (N \bmod 8)$, seguido (cuando aplica) de una reducción horizontal, y finalmente un bucle escalar de cierre que procesa uno a uno los elementos restantes hasta $i = N$. Este diseño garantiza correctud para cualquier valor de N , incluyendo N no múltiplo de 8. Véase el diagrama de flujo de control (archivo de diagrama, Sección I) para la representación gráfica completa de este patrón.

La estructura lógica completa de este patrón (verificación de múltiplos de 8 con `and ecx, ~7`, bucle AVX2, reducción horizontal y bucle remanente escalar) se documenta formalmente en el archivo `Docs/Diagrama de flujo de control_AVX2.pdf`. La asignación detallada de los registros vectoriales `ymm0–ymm6` y escalares auxiliares en cada fase del cálculo se describe en `Docs/Tablas de Asignación de Registros AVX2.pdf`.

V. CASOS DE PRUEBA

La Tabla I resume los casos borde obligatorios según la especificación del proyecto, verificados con `tools/verify_reference.py` contra una referencia en Python/NumPy con tolerancia relativa de 1×10^{-4} . Para cada tamaño se generó el archivo de entrada con `tools/gen_input.py`, se ejecutaron ambos kernels (`./bin/norm_sclar` y `./bin/norm_vector`) y se comparó la salida contra la referencia campo por campo (suma, media, varianza, desviación estándar, mínimo y máximo).

TABLE I: Casos de prueba y resultado de verificación

Caso	N	Escalar	Vectorial
$N = 0$ (error controlado)	0	PASA	PASA
$N = 1$	1	PASA	PASA
N no múltiplo de 8 (chico)	7	PASA	PASA
N no múltiplo de 8 (grande)	1001	PASA	PASA
Múltiplo exacto de 8	16	PASA	PASA
Valores constantes ($\sigma = 0$)	20	PASA	PASA
Valores negativos/extremos	100	PASA	PASA

The screenshot shows the output of the `verify_reference.py` script for three different cases. Each case includes a table of statistical data (sum, mean, var, stddev, min, max) for both scalar and vectorial kernels, comparing them against a reference. All results are marked as 'OK'.

Fig. 2: Salida de `verify_reference.py` para $N = 16$ (múltiplo exacto de 8), $N = 7$ (remanente) y $N = 20$ (caso $\sigma = 0$)

Adicionalmente se verificaron de forma explícita los dos casos borde más delicados, $N = 0$ y $N = 1$ (Figura 3): para $N = 0$ el programa reporta todos los estadísticos en `0.0` sin intentar ninguna división (evitando la división por cero de la media y la varianza), y para $N = 1$ la varianza y la desviación estándar son correctamente `0.0` porque no hay dispersión posible con un solo dato.

Por último, la Figura 4 documenta los casos de valores negativos/extremos ($N = 100$, con mínimos y máximos de $\pm 1,000,000$) y de remanente grande ($N = 1001$, no múltiplo de 8), ambos con **RESULTADO GENERAL: PASA** al compararlos contra la referencia de NumPy.

VI. RESULTADOS DE RENDIMIENTO

La Tabla II presenta el tiempo promedio del kernel (30 repeticiones) para ambas versiones, medido con

```

$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/gen_input.py 0 data/vector.dat random
./bin/norm_scalar data/vector.dat data/salida_vector_escalar.dat 1
./bin/norm_vector data/vector.dat data/salida_vector_vector.dat 1
Generado 'data/vector.dat' con N=0, modulo=0
N = 0
Suma = 0.000000
Media = 0.000000
Varianza = 0.000000
StdDev = 0.000000
Minimo = 0.000000
Maximo = 0.000000
Tiempo promedio del kernel (1 rep.): 0.000115 ms (+/- 0.000000 ms)
N = 1
Suma = 0.000000
Media = 0.000000
Varianza = 0.000000
StdDev = 0.000000
Minimo = 0.000000
Maximo = 0.000000
Tiempo promedio del kernel (1 rep.): 0.000125 ms (+/- 0.000000 ms)
$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/gen_input.py 1 data/input_1.dat random
./bin/norm_scalar data/input_1.dat data/output_scalar_1.dat 1
./bin/norm_vector data/input_1.dat data/output_vector_1.dat 1
Generado 'data/input_1.dat' con N=1, modulo=0
N = 1
Suma = 60.125076
Media = 60.125076
Varianza = 0.000000
StdDev = 0.000000
Minimo = 60.125076
Maximo = 60.125076
Tiempo promedio del kernel (1 rep.): 0.000181 ms (+/- 0.000000 ms)
N = 1
Suma = 60.125076
Media = 60.125076
Varianza = 0.000000
StdDev = 0.000000
Minimo = 60.125076
Maximo = 60.125076
Tiempo promedio del kernel (1 rep.): 0.000178 ms (+/- 0.000000 ms)

```

Fig. 3: Ejecución del kernels vectorial y escalar para $N = 0$ (error controlado, sin división por cero) y $N = 1$ (varianza y desviación estándar en 0.0)

```

$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/gen_input.py 100 data/input_edge.dat edge
Generado 'data/input_edge.dat' con N=100, modulo=0
$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/gen_input.py 1001 data/input_1001.dat 1
Generado 'data/input_1001.dat' con N=1001, modulo=0
N = 1001
Suma = 2.997126117
Media = 2.997126117
Varianza = 5.6772216208
StdDev = 2.380781912
Minimo = 0.000000
Maximo = 10000000000
Tiempo promedio del kernel (1 rep.): 0.000225 ms
$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/verify_reference.py data/input_1001.dat data/output_1001.dat stats.txt
comparacion de resultados
referencia      obtenido      error (rel. resultado)
N               1001              0
suma            2997126117         0
media           2997126117         0
varianza        56772216208       0
stddev          2380781912        0
minimo          0.000000           0
maximo          10000000000        0
res              0.000000           0
ver              10000000000.000000 2.997126117e-08 0%
stddev          5677216740.000000 5677216208.000000 0.000000 0%
media           2997126117.000000 2997126117.000000 0.000000 0%
var              0.000000           0
res              10000000000.000000 10000000000.000000 0.000000 0%
max              99.931000         99.931000         2.500000e-10 0%
RESULTADO GENERAL: PASA
$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/gen_input.py 1001 data/input_1001.dat 1
Generado 'data/input_1001.dat' con N=1001, modulo=0
N = 1001
Suma = 2.997126117
Media = 2.997126117
Varianza = 5.6772216208
StdDev = 2.380781912
Minimo = 0.000000
Maximo = 10000000000
Tiempo promedio del kernel (1 rep.): 0.000207 ms
$ cd /home/victorlira/1901-1902/Proyectos/1901-1902/Proyectos de Computadores 1/Project_1/PM1_Arqui$ python3 tools/verify_reference.py data/input_1001.dat data/output_1001.dat stats.txt
comparacion de resultados
referencia      obtenido      error (rel. resultado)
N               1001              0
suma            2997126117         0
media           2997126117         0
varianza        56772216208       0
stddev          2380781912        0
minimo          0.000000           0
maximo          10000000000        0
res              0.000000           0
ver              10000000000.000000 2.997126117e-08 0%
stddev          5677216740.000000 5677216208.000000 0.000000 0%
media           2997126117.000000 2997126117.000000 0.000000 0%
var              0.000000           0
res              10000000000.000000 10000000000.000000 0.000000 0%
max              99.931000         99.931000         2.500000e-10 0%
RESULTADO GENERAL: PASA

```

Fig. 4: Verificación para $N = 100$ (valores extremos $\pm 10^6$) y $N = 1001$ (remanente grande, $1001 \bmod 8 = 1$)

`clock_gettime(CLOCK_MONOTONIC)` alrededor exclusivamente de las llamadas a las funciones NASM (sin incluir lectura/escritura de archivo).

La desviación estándar se calcula sobre las mismas 30 repeticiones: `driver.c` mide el kernel con `clock_gettime` en cada una de las 30 repeticiones internas (dentro de la misma invocación del proceso), y calcula la media y la desviación estándar muestral de esas 30 mediciones, imprimiéndolas como Tiempo promedio del kernel (30 rep.): X ms (+/- Y ms). El guión `tools/benchmark.py` corre ambos binarios para cada N , captura esos dos valores directamente de la salida del *driver* y los guarda en `data/benchmark_results.csv` (Tabla II), junto con el tamaño adicional $N = 50,000,000$ requerido por el enunciado.

A. Análisis con `perf stat`

Se ejecutó `perf stat -e cycles,instructions,cache-misses` sobre ambas

TABLE II: Tiempo del kernel (ms, media $\pm$ desviación estándar de 30 repeticiones) y speedup

N	Escalar (ms)	Vectorial (ms)	Speedup
100	0.0004 $\pm$ 0.0000	0.0002 $\pm$ 0.0000	2.54x
1,000	0.0039 $\pm$ 0.0000	0.0011 $\pm$ 0.0000	3.51x
10,000	0.0392 $\pm$ 0.0011	0.0108 $\pm$ 0.0000	3.62x
100,000	0.3042 $\pm$ 0.0792	0.1083 $\pm$ 0.0010	2.81x
1,000,000	2.5847 $\pm$ 0.6402	0.3598 $\pm$ 0.0353	7.18x
10,000,000	23.1217 $\pm$ 0.5655	8.3707 $\pm$ 0.3545	2.76x
50,000,000	118.2065 $\pm$ 3.2022	44.7153 $\pm$ 0.4397	2.64x

```

victorlira@victorlira-laptop:~/PM1_Arqui$ column -s, -t data/benchmark_results.csv
N      scalar_ms  scalar_ms_stddev  vector_ms  vector_ms_stddev  speedup
100    0.000404   1.6e-05           0.000159   2.7e-05           2.5408805031446544
1000   0.003912   1.8e-05           0.001116   2.4e-05           3.505376344086021
10000  0.039216   0.001111         0.010836   2.4e-05           3.619847619047619
100000 0.304219   0.079169         0.108274   0.000994         2.8097142434933597
1000000 2.584737   0.640182        0.359798   0.035326         7.183855941389335
10000000 23.121738  0.565514        8.370681   0.354511        2.762229022943295
50000000 118.206481 3.202228       44.715283  0.439666        2.6435364615717627
victorlira@victorlira-laptop:~/PM1_Arqui$

```

Fig. 5: Salida de `column -s, -t data/benchmark_results.csv` (medias, desviaciones estándar y speedup para los 7 tamaños) y corrida del caso $N = 50,000,000$ mostrando el formato (+/- ...ms) que imprime el *driver*

versiones para $N = 10,000,000$, con 5 repeticiones internas por binario (Tabla III). Como la CPU de prueba es híbrida, `perf` reporta los contadores separados por tipo de núcleo (`cpu_core` y `cpu_atom`); los valores de la tabla son la suma de ambos. En la corrida escalar, el sistema operativo programó el proceso enteramente en núcleos `cpu_core` (0% de tiempo en `cpu_atom`), mientras que en la corrida vectorial el proceso se repartió entre ambos tipos de núcleo (6.62% `cpu_atom` / 93.38% `cpu_core`), lo que introduce una fuente adicional de variabilidad al comparar tiempos absolutos entre ambas versiones.

TABLE III: Contadores de hardware para $N = 10,000,000$

Métrica	Escalar	Vectorial
Instrucciones	≈1,637 M	≈567 M
Cache-misses	≈11.7 M	≈19.5 M
IPC (instr./ciclo)	2.22	0.78–0.88

Si bien la versión vectorial ejecuta aproximadamente 3 veces menos instrucciones que la escalar, la cantidad de fallos de caché es del mismo orden de magnitud en ambas versiones, ya que las dos deben transferir el mismo volumen total de datos desde memoria principal. El IPC notablemente menor en la versión vectorial (0.78–0.88 frente a 2.22 en la escalar) indica que el procesador permanece más tiempo detenido esperando datos de memoria por cada instrucción ejecutada. Esto es consistente con el patrón observado en la Tabla II: el speedup alcanza su punto máximo (7.18x) en $N = 1,000,000$, cuando el arreglo aún cabe en la jerarquía de caché, y cae

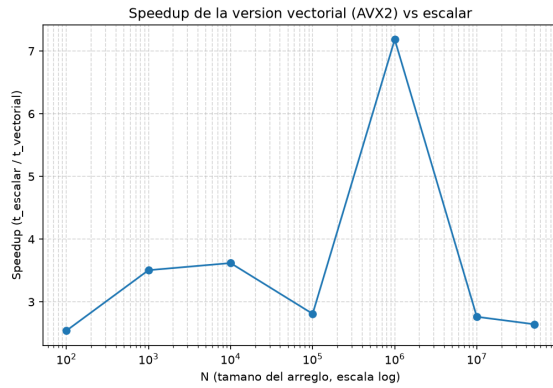


Fig. 6: Speedup de la versión vectorial (AVX2) respecto a la escalar, para los 7 tamaños de N (incluye $N = 50,000,000$)

a 2.76x en $N = 10,000,000$, cuando el arreglo (≈ 40 MB) excede la capacidad de la caché y el programa pasa de estar limitado por cómputo (*compute-bound*) a estar limitado por ancho de banda de memoria (*memory-bound*) [4]. El tamaño adicional $N = 50,000,000$ (≈ 200 MB, muy por encima de cualquier nivel de caché de la CPU de prueba) confirma esta tendencia: el speedup se estabiliza en 2.64x, muy cercano al 2.76x de $N = 10,000,000$, en vez de seguir cayendo, lo que es consistente con un régimen ya plenamente limitado por ancho de banda de memoria en lugar de seguir degradándose. Este comportamiento explica, además, por qué el speedup real casi nunca alcanza el límite teórico de 8x de AVX2.

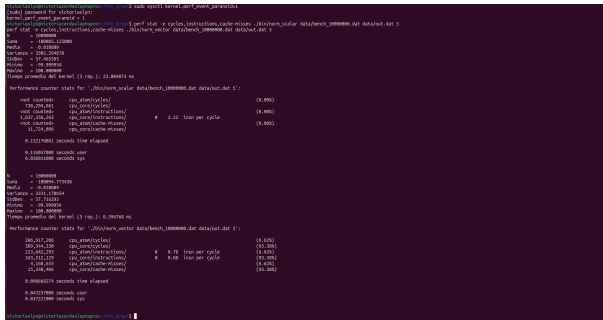


Fig. 7: Salida completa de `perf stat -e cycles,instructions,cache-misses` para `norm_scalar` y `norm_vector` con $N = 10,000,000$, 5 repeticiones

VII. EVIDENCIA DE LA SESIÓN DE GDB

Para la implementación escalar, se utilizó la interfaz visual interactiva de GDB (modo TUI) con el objetivo de inspeccionar el estado arquitectónico y el correcto paso de argumentos bajo la convención System V ABI. Se estableció un punto de interrupción en la función `sum_array`.

Listing 6: Comandos de GDB utilizados (Versión Escalar)

```
1 (gdb) break sum_array
2 (gdb) run
```

```
3 (gdb) tui reg vector ; Cambia a la vista de registros
vectoriales (XMM)
4 (gdb) x/4fw $rdi ; Inspeccion de los primeros 4
floats en memoria
5 (gdb) stepi ; Ejecucion paso a paso
```

Al alcanzar la instrucción `movss xmm1, [rdi + rax*4]`, se verificó mediante el comando `x/4fw $rdi` que el registro `rdi` apuntaba correctamente a los datos en la memoria física. Tras ejecutar `stepi`, se observó la carga exitosa del primer valor flotante desde la memoria principal hacia el registro escalar de 128 bits `xmm1`, validando el correcto funcionamiento del direccionamiento indexado.

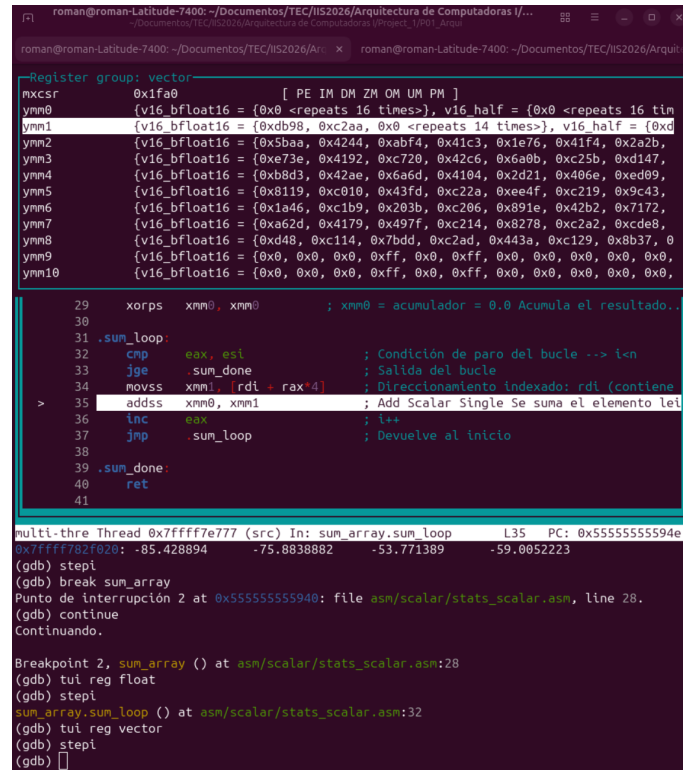


Fig. 8: Sesión de GDB (modo TUI) de la versión escalar: código fuente (centro), inspección de memoria (abajo) y actualización del registro de lectura `xmm1` (arriba).

Se colocó un punto de ruptura en la función `normalize_array` del kernel vectorial y se ejecutó el programa con un arreglo de prueba de tamaño $N = 16$:

Listing 7: Comandos de GDB utilizados

```
1 gdb --args ./bin/norm_vector data/input_small.dat \
2 data/output_debug.dat 1
3 (gdb) break normalize_array
4 (gdb) run
5 (gdb) stepi ; avanzar instruccion a instruccion
6 ; hasta el bucle vectorial (.
na_vec_loop)
7 (gdb) x/3i $pc ; confirma que la siguiente
instruccion
8 ; es vmovups [rsi+rax*4], ymm5
9 (gdb) print $ymm5.v8_float ; valor final normalizado, en
registro
10 (gdb) stepi ; ejecuta el vmovups (guardado en
memoria)
```

```
11 | (gdb) x/8fw $rsi      ; lee esos 8 floats directamente de
    |     memoria
```

Partiendo del breakpoint, se avanzó con `stepi` a través de `vbroadcastss`, `vucomiss` (chequeo de $\sigma = 0$) y las instrucciones `vsubps/vdivps` del bucle vectorial hasta llegar justo antes de `vmovups [rsi+rax*4], ymm5`. El comando `x/3i $pc` se usó para confirmar, desensamblando desde el contador de programa, que la próxima instrucción a ejecutar era exactamente ese `vmovups`. En ese punto, `print $ymm5.v8_float` mostró los 8 valores de punto flotante normalizados ($y_i = (x_i - \mu)/\sigma$) contenidos simultáneamente en el registro vectorial `ymm5`:

$$\{-1.19, 1.25, 0.97, -0.78, 0.05, -0.11, 0.58, 1.05\}$$

evidenciando que una sola instrucción SIMD opera sobre 8 datos a la vez. Tras ejecutar un `stepi` adicional (que realiza el `vmovups`), la inspección de memoria con `x/8fw $rsi` mostró exactamente esos mismos 8 valores ya escritos en el arreglo de salida (Figura 9), verificando la correcta transferencia de registro a memoria y, de forma cualitativa, que una única iteración del bucle vectorial produce en un solo paso lo que el bucle escalar tardaría 8 iteraciones en calcular.

[illegible]

Fig. 9: Sesión de GDB: `break normalize_array+run`, avance con `stepi`, `x/3i $pc` para confirmar la instrucción, `print $ymm5.v8_float` y `x/8fw $rsi` mostrando los mismos 8 valores en registro y en memoria

VIII. CONCLUSIONES, LIMITACIONES Y TRABAJO FUTURO

La vectorización SIMD con AVX2 ofrece mejoras de rendimiento significativas pero acotadas: el *speedup* observado va de 2.54x a 7.18x según el tamaño de entrada, alcanzando su pico (7.18x) en $N = 1,000,000$, cuando el arreglo aún cabe en la jerarquía de caché, y cayendo a 2.76x–2.64x para $N = 10,000,000$ y $N = 50,000,000$, cuando el volumen de datos (≈ 40 –200 MB) excede la caché disponible y el *speedup* se estabiliza en vez de seguir cayendo. El análisis con `perf stat` confirma que esta caída no se debe a un error de implementación sino a que el kernel pasa de estar limitado por cómputo a estar limitado por ancho de banda de memoria: la versión vectorial ejecuta ~ 3 veces menos instrucciones que la escalar, pero mueve el mismo volumen de datos desde RAM, por lo que su IPC cae a 0.78–0.88 frente al 2.22 de

la escalera. Esto explica, de forma general, por qué ninguna implementación real de este tipo de kernel alcanza el límite teórico de 8x de AVX2 para arreglos grandes.

Entre las limitaciones del estudio destacan: (1) la CPU de prueba es híbrida (núcleos *performance/efficient*), lo que añade variabilidad a las mediciones de `perf stat` y de tiempo de pared, según el tipo de núcleo en que el planificador del sistema operativo coloque el proceso en cada corrida; (2) no se probó AVX-512 por no estar disponible en el hardware utilizado; y (3) aunque la Tabla II ya reporta media $\pm$ desviación estándar (calculadas por `driver.c` sobre 30 repeticiones del kernel), esas 30 repeticiones ocurren dentro de una única invocación del proceso, por lo que no son completamente independientes entre sí: efectos como el estado de la caché o del planificador del sistema operativo pueden persistir de una repetición a la siguiente y correlacionar las mediciones, en lugar de que cada una parta de un proceso recién lanzado.

REFERENCES

- [1] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture*, Intel Corp., 2023.
- [2] M. Matz, J. Hubička, A. Jaeger, and M. Mitchell, *System V Application Binary Interface: AMD64 Architecture Processor Supplement*, v1.0, 2018.
- [3] The NASM Development Team, *NASM – The Netwide Assembler*, version 2.16 documentation, 2023. [En línea]. Disponible: <https://www.nasm.us/doc/>
- [4] A. Fog, *Optimizing subroutines in assembly language: An optimization guide for x86 platforms*, Technical University of Denmark, 2023.
- [5] R. Stallman, R. Pesch, and S. Shebs, *Debugging with GDB: The GNU Source-Level Debugger*. Free Software Foundation, 2023.